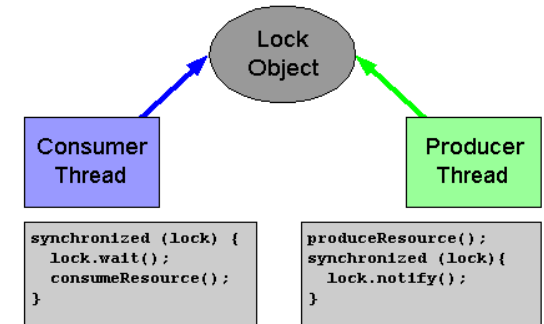




Høgskulen
på Vestlandet

DAT108 - Programmering og webapplikasjoner

F07 –Trådkommunikasjon, wait og notify/notifyAll



Lars-Petter Helland
Bergen
7. september 2026

Hva vi så på sist !! - 1

- › Hva **fleroppgavekjøring** (multithreading) er, og hvorfor dette er nyttig.
- › At vi kaller en "prosess" innad i et program for en **tråd**.
- › At et Java-program alltid har minst én tråd (**main**), men at vi kan opprette flere.
- › At en tråd i Java vil representeres av et **Thread**-objekt.
- › At "main" i en tråd er metoden **void run()**, som enten overkjøres i en **subklasse av Thread** eller i en **klasse som implementerer Runnable-kontrakten**.



Hva vi så på sist !! - 2

- › Hvordan vi
 - › **oppretter** | `new Thread()` / `new Thread(runnable)`
 - › **starter** | `t.start()`
 - › **pauser** | `Thread.sleep()`
 - › **stopper** | må lage en stopp-metode!
 - › og **venter på** | `t.join()` en tråd
- › Hvordan vi beskytter en "**kritisk seksjon**" i koden mot et "**race condition**" ved å bruke **synchronized**
 - › Enten på kodeblokker: **synchronized(lock)** { ... }
 - › Eller på hele metoder: **synchronized void m()** { ... }



Plan for i dag

- › En liten repetisjonsoppgave i fellesskap (fra konteeksamen v21)
- › Å vente på at noe skal skje i en annen tråd
- › Å sende signal til andre tråder om at noe har skjedd
- › `wait()`, `notify()`, `notifyAll()`
- › Tråders livssyklus
- › Flere Java-eksempler



Læringsressurser til i dag

Fra Java Concurrency and Multithreading Tutorial (<http://tutorials.jenkov.com/java-concurrency/index.html>):

- › Thread Signaling

Repetisjonsoppgave – Tråder basics – Oppgavetekst #1

Du skal lage et lite program som starter opp to tråder i tillegg til main-tråden, avslutter de på kontrollert måte, og avslutter main når de to trådene er ferdige.

Strukturen til main() ser slik ut:

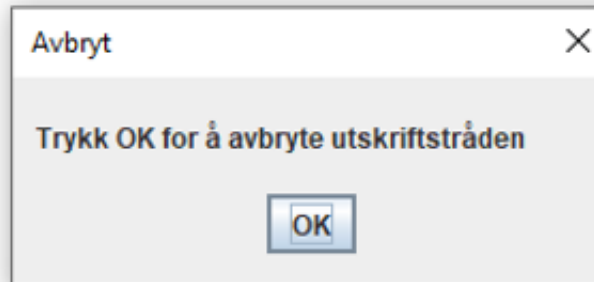
```
public static void main(String[] args) throws InterruptedException {  
  
    //Her opprettes og startes en tråd som går i en løkke og skriver ut  
    //en melding på skjermen ca. hvert sekund helt til tråden avsluttes  
    //på kontrollert måte.  
  
    //Her opprettes og startes en tråd som viser frem en JOptionPane meldings-  
    //boks. Når brukeren trykker på OK-knappen skal utskrifts-tråden få beskjed  
    //om å avslutte, og meldingsboks-tråden vil også være ferdig.  
  
    System.out.println("Tråder er startet. Venter på at de er ferdige ...");  
  
    //Her ventes det på at de andre trådene er ferdige før main avsluttes.  
  
    System.out.println("Begge tråder er ferdige!");  
    System.out.println("Main-tråd ferdig!");  
}
```



Repetisjonsoppgave – Tråder basics – Oppgavetekst #2

Kjøringen kan se ca. slik ut:

```
Tråder er startet. Venter på at de er ferdige ...  
Dette er en gjentakende melding !! :)  
Dette er en gjentakende melding !! :)  
Dette er en gjentakende melding !! :)  
Dette er en gjentakende melding !! :)  
Dette er en gjentakende melding !! :)  
Dette er en gjentakende melding !! :)
```



og etter at OK trykkes:

```
Begge tråder er ferdige!  
Main-tråd ferdig!
```

Meldingsboksen som venter på brukers OK kan fås frem slik:

```
JOptionPane.showMessageDialog(null, "Trykk OK for å avbryte utskriftstråden",  
                                "Avbryt", JOptionPane.PLAIN_MESSAGE);
```

Repetisjonsoppgave – Tråder basics - Oppgavetekst #3

IntelliJ-prosjekt i dag: 📁 f07-wait-notify

Pakke for eksempelet: 📦 dat108.eksv21.oppg5



- a) Skriv koden for utskriftsloop-tråden. Du må selv avgjøre om dette kan/skal være en Thread eller en Runnable, og om den skal være konkret eller anonym.
- b) Skriv koden for meldingsboks-tråden. Du må selv avgjøre om dette kan/skal være en Thread eller en Runnable, og om den skal være konkret eller anonym.
- c) Skriv koden i main som oppretter, starter og venter på disse trådene (slik det er vist i programkoden til main over).



Thread signaling

- › Det kan i en del sammenhenger være nyttig for tråder å kunne sende signaler til hverandre, f.eks. om at en tråd er ferdig med noe.
- › Vi skal begynne med en naiv implementasjon og deretter se på en mer kompleks mekanisme.
- › En enkel måte er at man sender signal via en verdi i et delt objekt, så kan tråder sjekke denne verdien jevnlig (løkke) og ta valg basert på det. Denne måten kalles "busy waiting" siden tråden aktivt sjekker en verdi.
- › En litt mer avansert måte er å bruke en innbygget mekanisme for å sende signaler.
- › Vi gjør litt Java ...



Eksempel - busy wait og wait/notify

IntelliJ-prosjekt i dag: 📁 f07-wait-notify

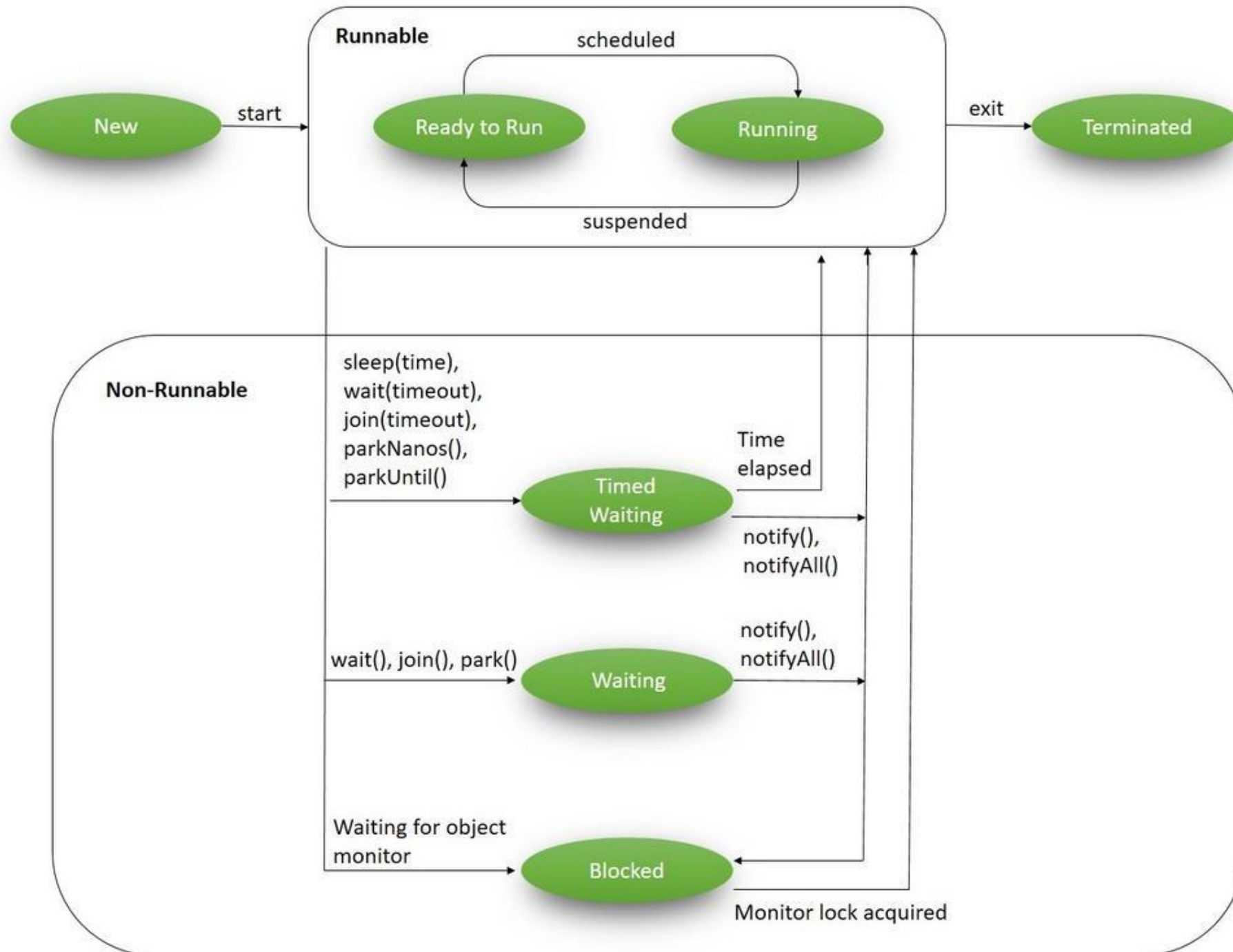
Pakker for eksempelet: 📦 busywait

📦 waitnotify



Vi starter helt fra scratch i dag. Løsning legges ut etter timen.

Livssyklus





Eksempel - wait/notify med meldingsobjekt

IntelliJ-prosjekt i dag: 📁 f07-wait-notify

Pakke for eksempelet: 📦 waitnotifymelding

Fortsetter fra kopi av forrige eksempel.



Eksempel - sender og mottaker

IntelliJ-prosjekt i dag: 📁 f07-wait-notify

Pakke for eksempelet: 📦 zendermottaker

Ikke gjøre. Kun vise og forklare.



Hva har vi sett på i dag

- › problemet med busy waiting
- › wait, notify, notifyAll for å få tråder til å samarbeide
- › en "blokkerende" meldingsklasse
- › ...



Videre

- › Lab:
 - › Lab - Oblig2 om tråder er lagt ut. Kort frist søndag 20.09.
- › Fredag :
 - › Vranglås, rettferdighet, ...
 - › Om låser (Lock, Condition, ReadWrite, reentrancy, ...)
 - › ...